Name: Tomas Gebrewold

WSU ID: 011822853

1. Introduction

This project implements a sequential HTTP proxy that handles **HTTP/1.0 GET** requests. The proxy sits between a client (browser) and an origin server, receives an HTTP request from the client, parses the URL, connects to the correct server, constructs a valid HTTP/1.0 request, forwards it, and relays the server's response back to the client. The purpose of the project is to gain experience in socket programming, request parsing, HTTP protocol behavior, and robust server design.

2. Design and Architecture

The proxy uses a standard accept-loop structure. For each client connection, the workflow is:

1. Accept a connection using accept().
2. Read the request line and headers using RIO.
3. Parse the method, URL, and HTTP version.
4. Extract hostname, path, and port using parse_url().
5. Connect to the destination server using socket() + getaddrinfo() + connect().
6. Build an HTTP/1.0 GET request with required headers.
7. Forward the request to the server using send().
8. Relay the response back to the client using recv() → send().
9. Close both sockets and wait for the next connection.

The code is organized into three main functions:

- **handle_client()**: reads and parses the client request.
- **handle_server()**: connects to the target server and returns a socket descriptor.
- **process_request()**: builds the request and forwards data between both sockets.

This separation keeps the implementation easy to understand, improves maintainability, and simplifies debugging.

3. HTTP Request Construction

All requests forwarded by the proxy follow the **HTTP/1.0** format regardless of the client's request version. The proxy must always include these headers:

Host: <hostname>
User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:10.0.3) Gecko/20120305 Firefox/10.0.3
Connection: close
Proxy-Connection: close

Additional client headers are forwarded unless they conflict with the above ones. The request ends with a blank line (\r\n). Binary response content is relayed directly using raw recv() and send() loops so that images, CSS, and other files transfer correctly.

4. Robustness and Error Handling

The proxy checks the return values of all socket functions and handles malformed requests and abrupt client disconnects. It supports port reuse with setsockopt(SO_REUSEADDR) and manages partial sends/receives for large responses. Fixed-size buffers are used conservatively to avoid memory errors, and all file descriptors are closed properly after use. The proxy continues serving new requests even if one request fails, ensuring stability during long runs.

5. Testing

The proxy was tested with several tools:

- **telnet**: used to send manual HTTP GET requests for debugging request parsing.
- **curl**: used with verbose flags to verify correct headers and response forwarding.
- **netcat (nc)**: used to examine the exact bytes forwarded by the proxy.
- **Firefox**: configured to use the proxy (127.0.0.1:<port>) to test real web traffic.
  HTTP-only websites such as **neverssl.com** were used to confirm proper behavior.

6. HTTPS Limitation

Modern websites primarily use HTTPS, which requires a CONNECT tunnel. This project only supports HTTP/1.0 GET requests, so HTTPS cannot be parsed or forwarded. When a browser attempts to access an HTTPS site through the proxy, it sends a CONNECT host:443 request, which the proxy does not implement. Because HTTPS traffic is encrypted end-to-end, the proxy cannot interpret it without full tunneling support. Failing on HTTPS traffic is expected and consistent with the assignment requirements.

7. Conclusion

The completed proxy meets all functional requirements of the assignment. It correctly handles sequential HTTP/1.0 GET requests, rewrites and forwards headers, manages socket communication between client and server, and safely relays HTTP responses of any size or type. Although it does not support HTTPS or concurrency, the implementation follows good software engineering practices, handles errors gracefully, and demonstrates a clear understanding of HTTP and TCP socket programming.